firmware\cube.cpp

```cpp
// ============================================================
// Prototype v3 - child Node
// ATtiny1616
//
// PlatformIO structure:
//   include/config.h
//   include/wire_bus.h
//   src/wire_bus.cpp
//   src/main.cpp
// ============================================================

#include <Arduino.h>
#include <avr/io.h>
#include <util/delay.h>
#include <stdint.h>
#include <stdbool.h>

#include "config.h"
#include "wire_bus.h"

// ------------------------------------------------------------
// Wire descriptors
// ------------------------------------------------------------

// Upstream orientation ports.
// Index in this array equals orientation value.
static const wire_t upstream[] = {
    { &PORTA, PIN1_bm, 1 },     // PA1 — orientation 0
    { &PORTA, PIN3_bm, 3 },     // PA3 — orientation 1
    { &PORTA, PIN4_bm, 4 },     // PA4 — orientation 2
    { &PORTA, PIN5_bm, 5 },     // PA5 — orientation 3
};

```

```
34 //suggestion: replace UPSTREAM_COUNT with upstream_count to avoid hardcoding the count
35 static const uint8_t upstream_count = sizeof(upstream) / sizeof(upstream[0]);
36
37 static const wire_t child_0 = { &PORTB, PIN0_bm, 0 };  // PB0 — child port 0
38 static const wire_t child_1 = { &PORTB, PIN1_bm, 1 };  // PB1 — child port 1
39
40 static const wire_t *children[CHILD_COUNT] = {
41     &child_0,
42     &child_1
43 };
44
45 // ------------------------------------------------------------
46 // Release all communication lines
47 // ------------------------------------------------------------
48
49 static void release_all_lines(void)
50 {
51     for (uint8_t i = 0; i < upstream_count; i++) {
52         wire_release(&upstream[i]);
53     }
54
55     for (uint8_t i = 0; i < CHILD_COUNT; i++) {
56         wire_release(children[i]);
57     }
58 }
59
60 // ------------------------------------------------------------
61 // Fast upstream scan
62 // scans all upstream lines for LOW and returns index of first detected LOW line (orientation value), or -1 if no line is LOW
63 // ------------------------------------------------------------
64
65 static int8_t scan_upstream(void)
66 {
67     for (uint8_t i = 0; i < upstream_count; i++) {
68         if (!is_line_high(&upstream[i])) {
69             return (int8_t)i;
```

```
70          }
71      }
72
73      return -1;
74  }
75
76  // ------------------------------------------------------------
77  // Wait for upstream command
78  // ------------------------------------------------------------
79
80  static bool wait_for_valid_upstream_cmd(
81      const wire_t **active_line,
82      uint8_t *orientation_mask
83  )
84  {
85      while (1) {
86          int8_t ori = scan_upstream(); // detects orientation line - 0/1/2/3 or -1 if no low is detected
87
88          if (ori < 0) {  //if no line is low, repeat orientation scan
89              continue;
90          }
91
92          const wire_t *w = &upstream[ori]; //assignment (zuordnung) of detected orientation line/pin to w
93
94          uint8_t b = 0;
95
96          if (!rx_byte_data_frame(w, &b)) { //receiving of message (sent bits) without start bit, false if no valid bits are received
    within timeout
97              continue;
98          }
99
100         //checks if received byte is CMD_STATUS, if not, repeat orientation scan and go into loop
101         if (b != CMD_STATUS) {
102             for (uint8_t i = 0; i < upstream_count; i++) { //release all upstream lines
103                 wire_release(&upstream[i]);
104             }
```

```
105             wire_release(&child_0); //release child lines, just in case
106             wire_release(&child_1); //release child lines, just in case
107             continue;
108         }
109 
110         // if correct CMD_STATUS is received, ausgabeparameter aus funktion: assign detected line to active_line and orientation
    value to orientation and return true
111         *active_line = w;
112         *orientation_mask = (uint8_t)ori;
113 
114         return true;
115     }
116 }
117 
118 // ------------------------------------------------------------
119 // Poll one child port
120 // ------------------------------------------------------------
121 
122 static bool poll_child(
123     const wire_t *w,
124     uint8_t *child_payload,
125     uint8_t max_child_payload_len,
126     uint8_t *child_payload_len
127 )
128 {
129     uint8_t b = 0;
130 
131     /*
132      * 1. Send CMD_STATUS to child.
133      */
134     tx_byte(w, CMD_STATUS);
135 
136     /*
137      * 2. Switch direction / wait before receiving.
138      */
139     _delay_us(TURNAROUND_US);
```

```
    // wire_release(w);  redundant

    /*
     * 3. Wait for ACK_BUSY.
     */
    if (!rx_byte(w, &b, CHILD_TIMEOUT_US)) {
        return false;
    }

    if (b != ACK_BUSY) {
        return false;
    }

    /*
     * 4. ACK received, now receive full DATA_FRAME.
     *    rx_frame validates SYNC, LEN and CHECKSUM.
     *    It returns only the child payload.
     */
    if (!rx_frame(
            w,
            child_payload,
            max_child_payload_len,
            child_payload_len,
            FRAME_TIMEOUT_US
        )) {
        return false;
    }

    return true;
}

// ---------------------------------------------------------
// Arduino setup
// ---------------------------------------------------------

void setup()
```

```
{
    release_all_lines();

    Serial.begin(9600);
    Serial.println("Prototype v3 recursive node started");
}

// ------------------------------------------------------------
// Main loop
// ------------------------------------------------------------

void loop()
{
    /*
     * These variables must be static because orientation detection
     * should only happen once, but the detected active upstream line
     * must still be available in later loop() iterations.
     */
    static bool orientation_known = false;
    static const wire_t *active_upstream = nullptr; //code merkt sich dauerhaft die Orientierungsleitung
    static uint8_t orientation_mask = 0x00; //orientation mask,now 1 byte, later combined with child_mask to one byte
    /// create variables (LEN_gesamt, LEN_Loop child etc.)

    /*
     * 1. Detect orientation only once.
     */
    if (!orientation_known) {
        wait_for_valid_upstream_cmd(&active_upstream, &orientation_mask); // selects active upstream line and orientation value
(orientation value must be transformed to orientation_byte for sending )
        orientation_known = true;
    }  else {
        // if orientation is already known, no need to check orientation just receive CMD_STATUS
        uint8_t b = 0;

        if (!rx_byte(active_upstream, &b, PARENT_TIMEOUT_US)) {
            Serial.println("Timeout: no byte received");
```

```
            return;
        }

        if (b != CMD_STATUS) {
            Serial.println("Unexpected byte received");
            return;
        }
    }
    /* you can also do create a own function instead:
        static bool wait_for_cmd_on_known_upstream(const wire_t *w)
    {
        uint8_t b = 0;

        if (!rx_byte(w, &b, PARENT_TIMEOUT_US)) {
            return false;
        }

        return (b == CMD_STATUS);
    }*/

    /*
     * Safety check.
     * Should normally never be true after orientation_known == true.
     */
    if (active_upstream == nullptr) {
        orientation_known = false;
        return;
    }

    /*
     * 2. Send ACK_BUSY upstream on the detected active line.
     */
    _delay_us(TURNAROUND_US);
    tx_byte(active_upstream, ACK_BUSY);
    //wire_release(active_upstream);

```

```
247      /*
248       * 3. Poll own children with retries.
249       *
250       * Child payloads are stored separately first.
251       * child_payload_len[i] tells how long child i's payload is.
252       * child_done[i] tells whether child i has valid stored payload.
253       */
254      uint8_t child_payload_buf[MAX_PAYLOAD_LEN];                  // stores all received child payloads compactly - in no correct
    order
255      uint8_t child_payload_buf_len = 0;                           //tells how many bytes child has sent
256
257      uint8_t child_payload_offset[CHILD_COUNT] = {0};              // tells where child[i] payload starts inside shared buffer
258      uint8_t child_payload_len[CHILD_COUNT] = {0};                 // temporary storage for child payload lengths ->
    child_payload_len[child_index] = length of payload received from child
259      bool child_done[CHILD_COUNT] = {false};                      // stores if child has already responded and can be skipped
260
261      // poll children with retries
262      for (uint8_t attempt = 0; attempt < CHILD_RETRIES; attempt++) { //loop for every retry
263          for (uint8_t i = 0; i < CHILD_COUNT; i++) { //loop for every child
264
265              if (child_done[i]) {   // if child[i] has already responded -> true -> skip
266                  continue;
267              }
268              /*
269               * Temporary receive buffer for one child.
270               * The child frame is received here first.
271               */
272              uint8_t temp_payload[MAX_CHILD_PAYLOAD_LEN];        //temporary payload of children is written in the temp_payload
273              uint8_t temp_payload_len = 0;
274
275              if (!poll_child(                //polls children
276                      children[i],             //wire/pin of current child
277                      temp_payload,            // where received payload should be stored temporarily
278                      MAX_CHILD_PAYLOAD_LEN,   //max payload bytes child is allowed to send
279                      &temp_payload_len        // adress of variable where poll_child() writes actual received payload length
280                  )) {
```

```
281                 continue;
282             }
283 
284             /*
285              * Check if this child payload still fits into the shared flat buffer.
286              */
287             if ((uint16_t)child_payload_buf_len + temp_payload_len > MAX_PAYLOAD_LEN) {
288                 continue;
289             }
290 
291             /*
292              * Store start position and length for this child.
293              */
294             child_payload_offset[i] = child_payload_buf_len;            //says where child payload should be stored to be in the
    correct order
295             child_payload_len[i] = temp_payload_len;
296 
297             /*
298              * Copy temporary payload into the shared flat buffer.
299              */
300             for (uint8_t j = 0; j < temp_payload_len; j++) {
301                 child_payload_buf[child_payload_buf_len++] = temp_payload[j];
302             }
303 
304             /*
305              * Mark this child as done.
306              */
307             child_done[i] = true;
308         }
309     }
310 
311     /*
312      * 4. Build own/nodes payload:
313      *
314      * Payload format:
315      *   NODE_ID ORIENTATION CHILD_MASK CHILD_PAYLOADS...
```

```
316      *
317      * The parser later expects child payloads in child-port order.
318      */
319     uint8_t payload[MAX_PAYLOAD_LEN]; //final payload to be sent upstream
320     uint8_t payload_len = 0;          //length of final payload
321     uint8_t child_mask = 0x00;        //child_mask to be sent in final payload
322
323     //stores own payload into the first payload position, ++ = incresas payload_len after writing, so next byte will be written on
    the next position
324     payload[payload_len++] = NODE_ID; //writes NODE_ID into the first free payload position
325     payload[payload_len++] = orientation_mask; //writes orientation mask byte into the next free payload position
326     payload[payload_len++] = 0x00;   // placeholder for child_mask
327     // e.g payload[0] = NODE_ID, payload[2]= 0x00; payload_len = 3
328
329     //in a loop add children payloads to the final payload
330     for (uint8_t i = 0; i < CHILD_COUNT; i++) {
331         if (!child_done[i]) {  //if child[i] didn´t respond, skip it
332             continue;
333         }
334
335         uint8_t offset = child_payload_offset[i];
336         uint8_t len = child_payload_len[i];
337
338         /*
339          * Check whether this child payload still fits into the final payload.
340          * If it does not fit, do not set the child bit.
341          */
342         if ((uint16_t)payload_len + len > MAX_PAYLOAD_LEN) {
343             continue;
344         }
345
346         //create child_mask
347         child_mask |= (1 << i);   //set bit for child[i]n in child_mask (LSB = child 0)
348
349         /*
350          * Copy child i's payload from the flat buffer into the final payload.
```

```
351          */
352         for (uint8_t j = 0; j < len; j++) {
353             payload[payload_len++] = child_payload_buf[offset + j]; //copies stored payload into final payload and increases
    payload_len for next byte
354         }
355     }
356 
357     /*
358      * Now write final child_mask into own node header.
359      */
360     payload[2] = child_mask;  //before was in payload[2] the placeholder 0x00
361 
362     /*
363      * 5. Send complete frame upstream.
364      */
365     _delay_us(TURNAROUND_US);
366     tx_frame(active_upstream, payload, payload_len);
367     wire_release(active_upstream);
368 
369     /*
370      * 6. Release all lines.
371      */
372     release_all_lines();
373 
374     /*
375      * 7. Debug output.
376      */
377     Serial.print("Sent payload_len = ");
378     Serial.println(payload_len);
379 
380     Serial.print("orientation = ");
381     Serial.println(orientation_mask);
382 
383     Serial.print("child_mask = 0b");
384     if ((child_mask & 0b10) == 0) {
385         Serial.print("0");
```

```
    }
    Serial.println(child_mask, BIN);
}
```